LAUREL

Mathematical Interfaces and Proof-Producing Residual Types

A concise mathematical proof language over Lean

Fourth iteration of the Brainstorm campaign

6 September 2026

The central proposal
A proof step should produce either evidence of its exact claim or a conditional proof explaining what remains to be established. Mathematical properties belong to a fixed object's usable interface; missing properties become explicit, composable obligations, not silent assumptions.

This article develops a property-aware typing discipline, an exact finite algorithm for alternative residual contracts, a repair rule for lost hypotheses, and a realizability-aware account of behavioral synthesis. It relates the design to both round-three syntheses, representative ProveIt and Fermat formalizations, and Leant's existing synthesis boundaries.

Evidence in the accompanying archive

An executed symbolic reference implementation, an independent derivation checker, exhaustive and adversarial tests, and generated ordinary-Lean conditional proof skeletons. The mathematical frontend and Lean integration remain proposed; no fresh Lean compilation or human-productivity result is claimed.

Prepared by OpenAI for Vladimir Reshetnikov

Abstract

Laurel is a proposal for mathematical proof authoring in which types expose the properties and behavioral laws needed by the next mathematical operation. Its foundation remains ordinary Lean: structures and meaning-bearing parameters are fixed, properties carry proofs, and every accepted assertion has a proof of the proposition actually requested. Its principal increment over the third brainstorming round is *proof-producing residualization*. Instead of merely failing when a premise is unavailable, elaboration constructs conditional proofs indexed by explicit residual requirements. In a finite ground Horn fragment, an antichain algorithm computes exactly the inclusion-minimal offered premise sets sufficient for a goal. We prove its relative completeness and conditional soundness, explain its exponential worst case, and provide a tested symbolic implementation. A separate transformation reopens used hypotheses after an edit without pretending that old evidence remains valid. For Leant, we generalize the affine-length example to a realized observation span: finite witness tests establish an affine universal law only when a coverage theorem connects those witnesses to all admissible inputs. Worked studies cover exact Frey quotients, sharp Fabius regularity, local differentiation, and almost-everywhere consumers. An elaborator extension is the recommended first implementation; a native refinement primitive is specified as a measurable alternative rather than dismissed or assumed necessary. The article distinguishes design, written mathematical proof, executed symbolic tests, and the kernel verification still to be done.

Contents

1 The fourth-round decision

1.1 What should no longer be redesigned

The two round-three syntheses reach substantially the same architectural conclusion. An author-facing object may acquire proved properties without changing its carrier or identity. Operations should request those properties through theorem-backed interfaces. Behavioral contracts belong to the actual function and its actual arguments. A computer-algebra answer must preserve its requested relation and guards. Neither a candidate's type nor a successful callback is automatically a proof of its intended behavior. The reports also converge on an empirical warning: these principles do not establish that a separate language helps its users [1, 2].

Laurel accepts this foundation. It does not claim to discover refinements, dependent contracts, proof-producing automation, observation interfaces, or an obligation display. In particular, the first synthesis already proposes an *obligation frontier*, and the second explicitly asks for an implemented inference engine instead of another collection of keywords. Their strongest criticism is well taken: composing already-correct theorem applications proves a small logical lemma, but does not prove the correctness or usefulness of an elaborator.

The next useful design decision is therefore operational:

> **A partially successful mathematical step is an artifact, not just an error.**
> For a fixed target G, retain a conditional proof $\Delta \to G$, where Δ states exactly the remaining premises of the selected route. Several routes may have different residual contracts. A route is complete only when its residual contract is discharged in the current context.

The arrow here is an abbreviation for a dependent telescope when necessary. It does not authorize adding assumptions to a published theorem. It gives the editor and the synthesizer something precise to compose, explain, cache, repair, and eventually close.

1.2 Four contributions and their status

This proposal makes four commitments beyond restating the shared architecture. First, it gives an exact finite semantics and an executable algorithm for alternative residual contracts, rather than leaving requirement inference at the level of policy. Second, it makes a lost-premise repair operation explicit: abstract the missing hypothesis out of the old derivation and produce a new conditional one. Third, it gives a domain-sensitive generalization of the affine-length bridge using the span of *realized* observations, including correlated inputs. Fourth, it specifies the boundary between these services and an actual Lean extension, including the conditions under which a more invasive type-system experiment would be worth attempting.

Only the symbolic residual engine, its checker, its narrow repair operation, and its tests are implemented in this archive. The typing and coverage theorems below have written proofs. Generated Lean files are ordinary conditional proof skeletons, but a Lean executable was not available in this environment and they were not compiled. There is no new mathematical-language parser over Lean expressions, no theorem-registry integration, and no usability study. These limitations delimit the evidence; they do not prevent the design or mathematical arguments from being examined.

1.3 What "more natural" should mean

The target is not unrestricted natural language and not the shortest possible source file. The target is *mathematically useful compression*: the author names the objects, the claim, and the argument's

significant transitions; the system reconstructs routine proof obligations while preserving every distinction that affects meaning.

For example, "differentiate the identity here" can be natural when the identity is known in a neighborhood of the point. It is misleading when only equality at the point is known. "Transport the exact quotient" is useful compression when divisibility and the destination's invertibility requirement are retained. It is false compression when integer division has been silently read as field division. Laurel should shorten the work needed to establish such premises, not hide that they matter.

2 Source diagnosis: what the examples actually cost

2.1 Reading scope and immutable revisions

Both main round-three synthesis texts were read closely, including their reconciliation, implementation questions, and evidence qualifications. This article is not a rerun of their audits. Direct source inspection was then used for representative mathematical and synthesis boundaries. Table 1 records the revisions; Appendix A gives the file locators and the scope of each observation.

Repository	Revision prefix	Use in this article
Brainstorm	`bf333390`	Both round-three syntheses, including reconciled corrections.
ProveIt	`24ce8bd7`	Fabius regularity theorem implementations.
anthropics/fermats-last-theorem	`aa2d8b34`	Exact integral-to-rational Frey-model transport.
Leant	`3a40904b`	Remote main observed here; README, synthesis internals, Length contract source.

Table 1: Source revisions are identities, not claims that the repositories were rebuilt. Full revisions are retained in the archive.

There is a version distinction worth preserving. The syntheses also discuss a Leant snapshot beginning `823259f7`, with additional observations about exact behavioral checking. The remote main returned during this review was `3a40904b`. Results attributed to the newer inspected snapshot below are explicitly attributed to the syntheses, not relabeled as observations of the remote main. Likewise, the syntheses' Lean 4.32.0 checks and the FLT repository's toolchain are not one shared environment [1, 2, 5].

2.2 Fabius regularity: mathematical facts versus representation work

The directly inspected `Regularity.lean` establishes global Lipschitz bounds for the bounded Fabius function and Rvachev's up function, then separately proves that the constant 2 is least. The mathematical shape is compact: a derivative formula and a range bound imply a derivative norm bound; a mean-value theorem gives Lipschitz continuity; derivative attainment at a midpoint supplies the lower bound on every possible constant [3].

The code also performs work that a mathematician usually compresses: specializing a range theorem at a transformed argument, turning a derivative certificate into an equality involving the derivative operator, normalizing absolute values, converting between a real bound and a nonnegative-real constant, and presenting a metric statement as an absolute-difference inequality. These are plausible interface obligations. Their repetition suggests reusable adapters, not a defect in the underlying logic.

Several proofs in the same file are already short. An endpoint majorant follows quickly from the Lipschitz estimate at x and 0; the final leastness theorem packages an upper and a lower bound. Any honest

comparison must keep these as controls. A new surface should not be credited with a compression obtained entirely by moving a theorem into a library that ordinary Lean could use too.

2.3 Frey coefficients: the type of division matters

The inspected FLT solution proves that an integral Weierstrass model maps to its rational counterpart. Its nontrivial coefficient obligations concern

$$a_2 = \frac{b^p - 1 - a^p}{4}, \qquad a_4 = \frac{-a^p b^p}{16}.$$

In the integral model the divisions are integer divisions; in the rational model they are field divisions. The proof establishes divisibility before using a rational cast-of-division lemma, and separately reduces the other coefficients by extensionality [4].

This is exactly where richer interfaces can help. A value known as an *exact quotient* carries a balance equation or divisibility evidence. Its transport through a ring map preserves the balance equation. An inverse formula requires the image of the denominator to be invertible. These are three related but different guarantees. The desired concise step is not "casts commute with division"; it is "transport this exact quotient along this map under this destination condition."

The file also contains discriminant and c_4 calculations, coprimality arguments, and valuation uses. Not all of its length belongs to the one theorem its filename names. Laurel's evaluation should isolate the actual declaration and its dependencies rather than using a whole generated file as a denominator for a compression percentage.

2.4 Leant is more than bare type inhabitation

Leant's current-tree documentation describes candidate ranking, exact rendering, retained provider identities, candidate/evidence association, verification boundaries, and separate negative-outcome categories. The Length contract module itself says that its laws are passive assertions supplied by an integration layer; the handoff checks identities and candidate correspondence rather than deriving the behavioral law from the provider's name or implementation. It also has a joint two-output contract vocabulary [6, 7, 8].

Laurel should build on that separation, not caricature Leant as returning arbitrary well-typed terms. Its proposed contribution is an acceptance target with explicit behavior, residual obligations, and retained proof evidence. A nominal association seal is valuable engineering, but the bridge from an abstract provider law to a theorem about the exact Lean provider remains a separate semantic requirement. Source inspection here is not a new Leant regression run.

2.5 A cost taxonomy for the language experiment

The observed costs fall into four classes. *Mathematical premises* include divisibility, locality, integrability, and attainment; removing them changes the claim. *Reusable interface work* includes cast adapters, bundled projections, and theorem composition. *Search work* is finding the right theorem and its instantiation. *Presentation and environment work* includes notation, source packaging, and structure management. Laurel addresses all four, but its success must be measured separately for each. A library improvement is valuable even when it gives no evidence for a new parser.

3 The proposed authoring experience

3.1 A small mathematical surface

The following is proposed Laurel notation, not the syntax of the executed symbolic prototype:

```
theorem quotient_derivative
  given f, g : Real -> Real and x : Real
  assume f is differentiable at x
  assume g is differentiable at x
  assume g(x) > 0
  show D(fun t => f(t) / g(t))(x)
       = (D(f)(x)*g(x) - f(x)*D(g)(x)) / g(x)^2
proof
  by quotient_rule
```

The operation identifies a theorem interface, not a string-to-tactic macro whose success may change the target. Its nonzeroness demand is supplied from the positivity proof. The author need not write the intermediate lemma, but its instantiated proposition and proof remain available.

Now delete the positivity assumption. Laurel should keep the exact displayed theorem target and produce something like:

```
Unfinished: quotient_rule needs g(x) != 0.
Conditional route retained:
  g(x) != 0  ->  the requested derivative identity.
Other registered sufficient routes:
  g(x) > 0
  g(x) < 0
No theorem has been published under an extra assumption.
```

The three routes are alternatives, not a conjunction. The direct nonzeroness route is logically weaker than either sign condition, but a sign condition may be easier to obtain from the current mathematical context. Inclusion-minimal support and logical weakness are deliberately not conflated.

This example does not itself justify a new language: a library wrapper could make ordinary Lean equally short. Its purpose is to define what happens during failure and repair, where a conditional proof can be more useful than a tactic trace ending with an unexplained subgoal.

3.2 Five actions, with distinct logical effects

Laurel needs only a small set of essential actions. A `given` binder introduces data; `assume` introduces a hypothesis in an explicitly scoped context; `establish` asks for a proof about already fixed data; `construct` delegates an authorized witness problem; and `show` fixes a proposition to be proved. A `by` clause can select a required method, whereas `by?` can be a search request. Spelling is secondary; the distinctions are not.

For instance, `establish a is positive` never replaces a with a positive number. In contrast, `construct a positive real` authorizes choosing a witness satisfying that specification. Normalizing a triple by dividing by a common factor similarly constructs new data. The old and new triples are related by a theorem; they are not the same object with an extra label.

3.3 A type hover should explain an interface

For a real function f, a compact view might read

$$f : \mathbb{R} \to \mathbb{R} \quad \text{with} \quad \text{Differentiable}(f), \quad \text{Lipschitz}(2, f), \quad f(0) = 0.$$

This is not a new intersection of unrelated Lean carrier types. It is the same elaborated function with three proofs in the local evidence index. Opening the view should reveal the exact predicates, the domain, the chosen structures, and the proofs or residual requirements that support them. A pending claim must be visually different from an established property.

The default mathematical view must expose conditions that alter interpretation: a measure, a neighborhood, a branch of a root, a finite precision, an admissibility guard, or whether a solution set is complete. Routine proof terms can be folded; the guarantee cannot. Every displayed assertion, even one unused by the final proof, must have its own accepted evidence before a document is called checked.

4 Property-aware types without changing objects

4.1 The three layers of a mathematical interface

A Laurel interface has three layers:

$$\underbrace{t : A}_{\text{data and carrier}} \quad ; \quad \underbrace{\Sigma}_{\text{fixed structure arguments}} \quad ; \quad \underbrace{\rho(t)}_{\text{proved or requested propositions}} \quad .$$

The first two determine meaning. The last describes available evidence and demands. The structure layer includes not merely a type name but the ring operations, scalar action, topology, measure, order, and other parameters actually used in the predicates. Their printed names need not all appear in prose, but their identities must be fixed in the elaborated request.

An *anchored refinement* $t : A \mid \rho$ therefore means that $t : A$ and the propositions in $\rho(t)$ have evidence. It does not mean that t has been rebound to a new subtype variable. A package exported through an API can of course use a subtype or a structure. Lean already supports dependent functions, propositions, and data-dependent types; the proposed change is which obligations the authoring layer knows how to request and present [9].

4.2 The conservative core rules

Let Γ be a well-formed Lean context with all meaning-bearing arguments fixed. The basic anchored rules are:

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash p : P(t)}{\Gamma \vdash t : A \mid P}, \qquad \frac{\Gamma \vdash p : P(t) \qquad \Gamma \vdash h : P(t) \to Q(t)}{\Gamma \vdash h(p) : Q(t)}.$$

Conjunction uses the ordinary pair of proofs. If $e : t = u$, property transport is equality elimination, producing a proof of $P(u)$ from one of $P(t)$. A weaker relation needs a consumer theorem. Equality of integrals from almost-everywhere equality, for example, is not equality elimination on the original functions.

These rules deliberately inherit the third-round calculus. Their value is not a new logical principle. Their implementation should make proof insertion, dependency accounting, and explanation uniform across otherwise unrelated mathematical APIs.

4.3 Proof-implicit requirements

An operation can declare a proof parameter as a requirement:

```
operation exact_quotient(n d : Int)
  requires d != 0
  requires d divides n
  returns q : Int with d*q = n
```

The exact semantics could be a chosen integer-division operation plus a theorem about its result, or an explicitly constructive quotient witness. The declaration must choose one. Its `requires` parameters elaborate to ordinary proof arguments; the property engine attempts to supply them from the local context, theorem applications, or an authorized external proof producer. A remaining proof argument is a residual obligation, never a successful argument of unknown truth.

A requirement is not automatically a type-class instance. Type classes remain appropriate for library APIs that expect them; a local adapter can introduce an instance when necessary. Registering every inferred fact as a global instance would mix data selection with proposition search and make the environment harder to predict.

4.4 Behavioral function types and variance

For a fixed total function $f : A \to B$, define

$$\mathrm{Beh}(f; P, Q) \;:\Longleftrightarrow\; \forall x : A,\ P(x) \to Q(x, f(x)),$$

where Q may relate the input to the output. From this contract, $P' \Rightarrow P$, and $Q \Rightarrow Q'$ on the relevant inputs and outputs, obtain $\mathrm{Beh}(f; P', Q')$. The precondition becomes stronger and the postcondition weaker; the underlying function does not change.

A function on the refined domain is different:

$$f : \{x : A \mid P(x)\} \to B.$$

It cannot be applied to arbitrary $x : A$ until a proof of $P(x)$ is supplied. Nor can it be extended to all of A merely by forgetting its domain. An extension needs an actual construction, a codomain inhabitant where appropriate, or an explicit noncomputable choice with its associated policy.

Relational properties require more than an output predicate. Injectivity, Lipschitz continuity, prefix locality, preservation of a diagram, and naturality quantify over multiple evaluations or multiple objects. Laurel permits arbitrary well-typed Lean propositions in a row. Its automatic fragment is much smaller than its specification language.

4.5 Composition retains the intermediate value

Suppose

$$\mathrm{Beh}(f; P, Q), \qquad \mathrm{Beh}(g; R, S),$$

and, for all relevant x, y, z, there are bridges

$$P(x) \wedge Q(x, y) \Longrightarrow R(y), \qquad P(x) \wedge Q(x, y) \wedge S(y, z) \Longrightarrow T(x, z).$$

Then $\mathrm{Beh}(g \circ f; P, T)$. To prove this, fix x with $P(x)$, obtain $Q(x, f(x))$, apply the first bridge to the *actual* $f(x)$, use the contract of g, and apply the second bridge with $y = f(x)$ and $z = g(f(x))$.

The premise $R(f(x))$ is often the missing obligation in a synthesized composition. Merely composing arrow types loses it. The residual engine should expose that premise at the retained intermediate value, including any equality or representation bridge used to identify it.

4.6 Refinement polymorphism rather than a hierarchy of wrappers

A reusable method should quantify over properties when its logic permits it. The consequence and composition theorems above are examples. A structure-preserving restriction can be parameterized by an arbitrary invariant predicate; an observation consumer can be parameterized by a relation together with its preservation theorem.

This is preferable to defining a separate carrier for every combination of smoothness, positivity, monotonicity, and integrability. A finite family of useful exported bundles can coexist with an open local evidence index. Proofs of equivalent propositions can be connected by an explicit equivalence theorem, but equivalent mathematical presentations are not assumed to have a common principal refinement type.

5 Residual types and the meaning of unfinished work

5.1 A judgment that returns a conditional proof

For a fixed proposition G, the central judgment is

$$\Gamma; \mathcal{R} \vdash G \rightsquigarrow (\Delta, \pi), \qquad \Gamma \vdash \pi : \Pi\Delta.\, G.$$

Here $\mathcal{R}$ is a selected, validated method profile and Δ is an ordered telescope of remaining proof obligations. In the flat fragment it is simply a finite set of propositions, interpreted as their conjunction. A completed result has an empty telescope and hence a proof of G in Γ.

The judgment makes a distinction that a mere list of unsolved goals does not: π records how solving those goals proves *this* original target. It is a proof continuation with an inspectable type. A user may save it as a conditional lemma, but doing so requires an explicit statement change or an explicitly conditional declaration. The original theorem remains unfinished until its own context discharges the telescope.

5.2 Terms that themselves need proof arguments

Not every construction produces a carrier term already available in Γ. Applying a function on a refined domain may require an unresolved proof. In that case the result is a term under the residual telescope,

$$\Gamma, \Delta \vdash t : A, \qquad \Gamma, \Delta \vdash p : P(t),$$

or equivalently a dependent function returning a refined result after its requirements are supplied. Laurel must not place this t into the completed data environment prematurely. The simpler anchored-view case, where $\Gamma \vdash t : A$ already holds and only $P(t)$ is pending, remains the common case for imported total Lean operations.

This distinction avoids an unsound erasure story. A proof parameter may be irrelevant to runtime computation while still being necessary for the term to be well-typed. Erasing a proof from generated machine code and omitting its obligation from a formal derivation are different operations.

5.3 Dependent obligations and authorized witnesses

An unresolved requirement may live under binders. For example,

$$\forall \varepsilon > 0,\ \exists N,\ \forall n \geq N,\ E(n, \varepsilon)$$

contains a witness N that may depend on ε. It must not be flattened into one global N or replaced by finitely many tested values of ε. Similarly, after constructing y with $Q(x, y)$, a later requirement $R(y)$ refers to that same witness, not to an independently synthesized value satisfying a similar type.

Laurel therefore separates *proof obligations* from *construction slots*. A construction slot has an authorized specification, such as $\exists y, Q(x, y)$ or a data-returning dependent pair. Its scope and quantifier order remain explicit. The finite antichain procedure below applies only after such data parameters are fixed. It does not reorder dependent telescopes or solve arbitrary existential synthesis.

5.4 Residuals are neither exceptions nor axioms

A timeout is an operational event, not a mathematical hypothesis. "The solver timed out" must never occur in Δ as though assuming it could prove the target. Likewise, a failed search for P is not a proof of $\neg P$.

Useful result states are: a proof of the fixed target; one or more conditional proofs; no route in a fully explored declared fragment; incomplete exploration with no route yet; a separately checked refutation; and an unsupported request. The middle two are not refutations. The UI can simplify their presentation, but the API must preserve these distinctions.

6 An exact finite algorithm for residual contracts

6.1 The grounded profile

Fix a finite set V of atoms, a set $K \subseteq V$ of known facts, a set $O \subseteq V$ of propositions allowed to appear as residual obligations, and a finite set of Horn rules

$$r: \quad p_1 \wedge \cdots \wedge p_j \Longrightarrow q, \qquad p_i, q \in V.$$

An empty premise list is permitted. Each atom denotes a fixed proposition in Γ; each rule is backed, in the intended Lean implementation, by a theorem instance in that same context. A local real variable can occur inside an atom. Grounding means its identity is fixed, not that it has been assigned a numerical value.

Let $\mathrm{Cl}_{\mathcal{R}}(S)$ be the least rule-closed set containing S. Define the *exact residual frontier* of q by

$$F(q) = \mathrm{Min}_{\subseteq}\big\{H \subseteq O : \ q \in \mathrm{Cl}_{\mathcal{R}}(K \cup H)\big\}. \tag{1}$$

$\mathrm{Min}_{\subseteq}$ removes strict supersets, not logically stronger formulas. If $\varnothing \in F(q)$, then $F(q) = \{\varnothing\}$ and the target is established in the profile. If $F(q)$ is empty, no offered premise set derives it within this profile.

The author or the operation chooses O. Offering the goal itself gives the valid but unhelpful conditional proof $q \to q$; it must be labeled as a restatement rather than progress. An unrestricted "assume anything" mechanism would make residualization vacuous.

6.2 Antichain operations

Represent each $F(q)$ as an antichain of subsets of O. For antichains A, B, define

$$A \oplus B = \mathrm{Min}_{\subseteq}(A \cup B), \qquad A \otimes B = \mathrm{Min}_{\subseteq}\{a \cup b : a \in A, \ b \in B\}.$$

Alternative derivations use $\oplus$; simultaneous premises use $\otimes$. The zero is the empty antichain and the multiplicative identity is $\{\varnothing\}$. These operations are a compact form of monotone Boolean provenance: a support H denotes the conjunction of its offered propositions, and an antichain denotes a disjunction of those conjunctions.

Algebraic provenance is not new. The use of addition for alternative derivations and multiplication for joint support is a central idea of provenance-semiring work [13, 14]. Laurel's proposal is to retain an actual conditional proof beside each support and expose the resulting alternatives as a mathematical typing and repair interface. It deliberately forgets proof multiplicity when choosing one representative for an identical support.

6.3 The procedure

Initialize an empty frontier at every atom. Insert $\varnothing$ at each $k \in K$ and $\{o\}$ at each $o \in O$. Repeatedly process every rule $p_1, \ldots, p_j \to q$. For every choice $H_i \in F(p_i)$, propose $H = \bigcup_i H_i$ at q. Reject it if an existing support is a subset of H; otherwise retain it and delete its strict supersets. Continue until no frontier changes.

For every accepted support, store a proof node. A known leaf refers to its proof in Γ. An offered leaf refers to the corresponding residual assumption. A rule node applies the registered theorem to its children's proofs. Nodes are immutable and refer only to already constructed nodes, so a cyclic rule graph never creates a cyclic proof object.

Theorem 6.1 (Conditional soundness)**.** *Suppose each known atom has a proof in Γ and each registered ground rule has a proof of its stated implication in Γ. Every support H retained for q by the procedure has a constructible proof*

$$\Gamma \vdash \left(\bigwedge_{h \in H} [\![h]\!] \right) \to [\![q]\!].$$

This holds even if the procedure is interrupted before reaching a fixed point.

Proof. For a known leaf use its existing proof, ignoring the empty conjunction. For an offered leaf project its proposition from the supplied conjunction. For a rule node, the union support contains every premise support. Project or weaken from the union's conjunction to each premise's conjunction, apply the children's conditional proofs, and then apply the rule theorem. This constructs the claimed term by induction on the finite proof DAG. Deleting another support does not change the validity of the retained node or of any earlier immutable node used by it. No fixed-point assumption enters this induction. □

Theorem 6.2 (Termination and relative completeness)**.** *With finite $V, O, \mathcal{R}$, the unbounded procedure terminates. On termination its frontier at every atom is exactly* (1)*. Fair processing order can affect chosen proof terms, but not the final support antichains.*

Proof. For an antichain A, let

$$\uparrow A = \{H \subseteq O : \exists a \in A,\ a \subseteq H\}.$$

Every successful insertion strictly enlarges $\uparrow A$, even when it removes supersets from the stored antichain. These upward-closed sets are subsets of the finite set $\mathcal{P}(O)$. Thus there are at most $|V|2^{|O|}$ strict enlargements over all atoms, and full passes eventually make no change.

Soundness of support membership in the set on the right of (1) follows either from the proof-node construction or directly by induction on insertions. For completeness, fix $H \subseteq O$. If $q \in \mathrm{Cl}_{\mathcal{R}}(K \cup H)$, it has a finite forward derivation. At initialization, every leaf in K is supported by $\varnothing \subseteq H$ and every leaf in H by its singleton. At the fixed point, induction on that derivation shows that every premise has some stored support contained in H. Applying the rule would insert their union or find a still smaller support already present. Consequently q has a stored support contained in H.

Now take a subset-minimal H satisfying (1). The stored support contained in it is itself sufficient, so minimality forces equality. Conversely, if a stored support were not subset-minimal among sufficient sets,

the preceding completeness argument would supply a smaller stored support, contradicting the antichain invariant. Equation (1) determines a unique antichain, which gives order independence at a completed fair fixed point. □

6.4 Why minimal supports are not weakest preconditions

The theorem is exact about a tuple $(V, K, O, \mathcal{R})$, not about mathematical implication in general. If O contains both $g(x) > 0$ and $g(x) \neq 0$, the singleton supports are incomparable by set inclusion even though one predicate implies the other. Keeping both is useful: a theorem elsewhere may readily establish the stronger-looking predicate.

A separate simplifier could remove logically dominated contracts, but only using proved implications and an explicitly chosen presentation policy. It would be a different optimization criterion. Laurel should never describe the surviving rows as the globally weakest hypotheses of the theorem. The known context may itself be inconsistent, and offered assumptions may be inconsistent; neither fact is detected by the finite Horn completeness claim.

6.5 Cycles and failures

With rules $P \to Q$ and $Q \to P$, no known facts, and no offered premises, both frontiers remain empty. With P offered, the procedure may return $P \to Q$ and $P \to P$. Neither result establishes P unconditionally. This simple case is a critical boundary test for conditional reasoning.

If no support exists at a completed fixed point, the exact statement is that no subset of the offered obligations derives the target by these rules. It does not imply that the target is false, unprovable in Lean, or impossible to establish by a theorem absent from the registry. A proof of that finite nonderivability could eventually be certified as a fact *about the profile*; it still would not be a refutation of the target proposition.

6.6 Cost, budgets, and deliberate incompleteness

The linear bookkeeping bound for ordinary finite fact closure does not extend to all alternative supports. Let each intermediate p_i follow either from a_i or from b_i, and let the goal require all p_i. With all a_i, b_i offered, the goal has 2^k incomparable supports, choosing one offered proposition for each i. Any explicit complete frontier must represent that output somehow.

The implementation in this archive uses a simple scan-and-antichain algorithm. It is suitable for specification and small tests, not a claim of an optimized editor engine. The width experiment reproduces the expected growth up to $1,024$ supports at $k = 10$. Larger systems may use decision diagrams, lazy route enumeration, support-width caps, or demand-specific ranking. Such choices must change the status to partial when they discard alternatives. A partial frontier still has sound conditional proofs by Theorem 6.1; it is not necessarily the complete or globally subset-minimal frontier.

A longer run need not preserve the exact list of displayed supports: a newly found smaller support may supersede a previous one. It does preserve the validity of previous proof objects and expands the upward-closed set of sufficient assumption sets represented by the search. That is the appropriate monotonicity claim.

7 From Lean theorems to a finite profile

7.1 The theorem registry is checked metadata

A registry entry should contain a theorem reference, universe instantiation, data-argument roles, premise roles, conclusion role, and a permitted matching pattern. None of these annotations is a new logical rule. Registration succeeds only after the declared shape is reconciled with the theorem's elaborated type. At use time the actual theorem application is type checked again.

For a quotient transport interface, metadata may identify the source numerator, source denominator, ring map, exactness premise, and destination-unit premise. It may not infer that a denominator is a unit merely because its source-language spelling looks like a positive integer. For an analytic interface, the region, filter, measure, and scalar field are ordinary meaning-bearing arguments, not optional decorations.

A rule can be logically valid while still unsuitable for automatic use. It may introduce an unconstrained function, produce an unbounded family of new terms, trigger expensive instance search, or hide the mathematical step the author intended. Registration therefore has two checks: the theorem is well typed, and its matching policy belongs to the chosen bounded profile. The latter is an engineering restriction, not a theorem about the proposition's truth.

7.2 A concrete bounded grounding policy

The first implementation should use a finite *term pool* extracted from the fixed goal, local facts, and explicitly selected method arguments. It includes their typed subexpressions and a bounded set of declared observations, such as the image of a denominator under a selected ring map. Predicate templates are instantiated only at terms in this pool.

A registration is automatic in this first fragment only when its data parameters are fixed by matching the requested conclusion or by selection from that finite pool. Newly generated terms must either already be in the pool or be produced by a constructor whose finite depth budget is explicit. The implementation also imposes budgets on candidate instances, atom count, and rule count. Exceeding a budget gives a partial profile with a visible explanation.

This closes a gap in a theorem that assumes finite grounding without saying how it happens. If there are n admissible terms, a registration with at most d independently enumerated data positions has at most n^d raw assignments before type and pattern filtering. This crude bound is not a performance estimate; it simply makes finiteness explicit. A theorem requiring synthesis of an arbitrary function is sent to an external construction service, not disguised as one more finite match.

Property propagation need not saturate the whole environment. Start from the consumer's demands, generate a bounded dependency slice, and then run the support algorithm on that slice. If a search service later proves a new atom at its fixed type, record the proof and create a new profile epoch. Completeness of the previous epoch refers only to its previous rules and facts.

7.3 Existing automation is a participant and a baseline

Mathlib's `fun_prop` already decomposes functions and applies registered property theorems; it also has transition rules and documents the search-space risks of those rules. It is therefore both a likely producer of evidence and a strong comparison point, not a capability Laurel may claim to invent [10].

Similarly, ordinary theorem application, simplification, arithmetic tactics, and `grind` can solve a fixed requested proposition. Laurel's service contract is simple: return a proof term of that proposition or an operational failure. A successful tactic does not need to expose a complete alternative-premise

frontier. A first version can accept it as a closed proof producer and reserve residualization for registered theorem interfaces whose premises are inspectable. The current Lean reference also documents `mvcgen` for verification conditions of monadic programs; mathematical contracts should interoperate with that separate effectful layer rather than pretending a pure predicate is a complete effect specification [11].

7.4 Meaning holes, proof holes, and witness holes

The frontend should classify unresolved variables by role before accepting automation. A *meaning hole* determines the statement: a scalar field, topology, region, measure, branch, precision, or quantifier-bearing object. It must be resolved by normal elaboration or by an explicit author choice before proof search. A *proof hole* requests evidence for a fixed proposition. A *witness hole* authorizes construction under a fixed specification.

This policy is not equivalent to forbidding all metavariables. Proof goals remain metavariables, and authorized synthesis needs data holes. The invariant is that a producer cannot instantiate an unauthorized data parameter merely to make the proposition easier. A concrete negative test should change an imported instance or leave a measure unspecified and check that the interface requests clarification rather than silently proving a different statement.

The symbolic prototype does not implement this classification: its atom strings are already fixed input. A real Lean prototype must test the invariant at elaboration snapshots, where a candidate can actually assign metavariables. This is one of the largest remaining gaps between the finite model and the intended language.

8 Repair, rewriting, and evidence lifetime

8.1 Lost hypotheses produce new conditional proofs

A common editing event is removal of a hypothesis that a previous proof used. Simply invalidating the entire step loses useful work; reusing the old proof unchanged is wrong. There is a precise middle operation.

Proposition 8.1 (Reopening a used premise)**.** *If a derivation in a fixed rule profile proves G from $\Gamma, h : P$, abstracting the assumption gives a derivation of $P \to G$ from Γ. For a finite set of removed known facts, the same transformation needs only those removed facts that actually occur as known leaves in the retained derivation.*

Proof. Replace each used removed-hypothesis leaf by a corresponding residual variable and rebuild the same theorem applications. Then abstract over those variables. Unused hypotheses occur at no leaf and need not be added to the new telescope. This is ordinary implication introduction, implemented on the derivation rather than justified by a cache identifier. □

The reopened support is sufficient, not necessarily minimal: another route may close the proof without it, and a fresh frontier computation may eliminate it. The archive implements this operation for the symbolic fragment. It requires identical atoms and identical rule declarations, deletes only named known facts, adds those facts to the offered set, creates a new context identity, reconstructs the proof DAG, and checks the new conditional derivation independently. It does not infer that arbitrary edited Lean contexts are equivalent. After more substantial edits, rebuilding the local evidence index is the safer initial policy.

8.2 Rewriting changes an anchor only through evidence

Suppose a property proof has type $P(t)$ and simplification replaces an occurrence of t with u. There are three distinct cases. Conversion may already let Lean regard the old proof as a proof of the new proposition. An explicit equality $t = u$ may permit equality transport. Or a theorem may relate the requested properties directly, such as $P(t) \leftrightarrow Q(u)$, without identifying the objects themselves.

An implementation must retain which case occurred. Matching pretty-printed normal forms is not evidence. For dependent indices, the predicate and its type arguments may also need transport. For different ring or topology structures on the same carrier, identical printed terms are especially misleading. Laurel should preserve the old anchor and insert a checked adapter when possible; it need not destructively rewrite the entire evidence row every time the goal changes.

8.3 Branch scope and publication scope

An assumption obtained in one branch is unavailable in its sibling. On leaving the branch it may be exported as an implication or as part of a case-elimination proof, not as an unconditional property. A frontier entry therefore has a local context, not merely a proposition string.

The intended implementation attaches evidence to Lean's current elaboration snapshot. On rebuilding a declaration, either reconstruct the index from the new local context or use a checked context embedding with typed substitutions. A persistent local identifier, source offset, or cryptographic hash is routing information; none is a proof of such an embedding. The symbolic context-mismatch tests check a deliberately weaker boundary: they reject differing exact profile identities.

8.4 Proof sharing and repair cost

Retaining a conditional derivation need not duplicate every proof term. Shared theorem applications can be represented by a DAG and emitted as local `have` declarations or equivalent shared expressions. The exporter in the archive does this for its small symbolic examples. Nevertheless, a large collection of alternative supports can retain many different derivations, and transports can enlarge proof terms even when the mathematical statement is unchanged.

A real implementation should record live index entries, retained proof nodes, generated term size, rebuild time, and the fraction of requests solved by a single theorem application. It should initially keep one derivation per identical support within a method profile. Preserving every stylistic route is a separate feature with separate cost; it is not needed for the support completeness theorem.

9 Behavioral synthesis and attainable observations

9.1 Three bridges, not one "verified" flag

For a synthesis request, fix a source specification $\mathrm{Spec}(f)$ and an authorized type for f. A successful candidate needs a proof of the specification about the exact accepted function. An abstract behavioral service may help find that proof, but three bridges are distinct: the candidate representation denotes the accepted source term; the provider laws used by the abstraction hold for their exact source providers; and the accepted abstract property implies the requested source property.

The Length contract source makes the second boundary particularly clear: the supplied provider law is an assumption of the abstract integration, not a theorem inferred from the provider name [8]. Laurel's

registration format would add a source theorem and a denotation bridge. A producer's operational receipt can accompany those proofs, but cannot replace them.

The newer Leant snapshot described in the syntheses checks exact candidate propositions and separately checks negations. That is stronger than treating type inhabitation as behavior. Laurel should preserve such direct checks as a valid route, and add retained proof evidence and semantic abstraction bridges where an abstract solver is used. The newer-snapshot claim here is the syntheses' observation, not an independent run in this work [1, 2].

9.2 Sound acceptance and sound rejection are asymmetric

Let A be the concrete input type, $P(x)$ its admissibility predicate, and $o : A \to M$ an observation map. A sound abstraction theorem may show that an accepted model certificate implies $\forall x, P(x) \to \mathrm{Spec}(f, x)$. It need not show that every true source specification can be recognized by the model.

For rejection, an abstract violating value m is not enough. One needs an admissible x with $o(x) = m$, together with the transfer direction that makes a source specification imply the model condition at m, or a direct proof that the source specification fails at x. The source domain may realize only a small part of M.

The standard stress case is `List Empty`. Its only element is the empty list, so its only realized length is 0. The constant-empty function preserves length on that type, although the two unrestricted affine models 0 and n have different coefficients. Over lists of natural numbers every nonnegative length can be realized by a list of zeros. Thus the same coefficient comparison has a different completeness story on the two domains. This distinction is central to the reconciled round-three reports [1, 2].

9.3 A generalization: the realized affine observation span

The domain issue can be stated more systematically. Suppose a fixed behavioral grammar has a sound observation theorem reducing its output property to an affine expression in rational-valued observations. Let

$$A_P = \{x : A \mid P(x)\}, \qquad o : A_P \to \mathbb{Q}^k, \qquad \widetilde{o}(x) = (1, o(x)) \in \mathbb{Q}^{k+1}.$$

Define the *attainable observation span*

$$W = \mathrm{span}_{\mathbb{Q}}\{\widetilde{o}(x) : x \in A_P\}.$$

This is not a claim that every vector in W is realizable. It is the linear space needed to reason about affine equalities on the realizable set.

Theorem 9.1 (Realized-basis certification)**.** *Let $u, v : \mathbb{Q}^k \to \mathbb{Q}$ be affine maps. Suppose admissible witnesses $x_1, \ldots, x_m$ satisfy the coverage theorem*

$$\forall x \in A_P, \quad \widetilde{o}(x) \in \mathrm{span}_{\mathbb{Q}}\{\widetilde{o}(x_1), \ldots, \widetilde{o}(x_m)\}. \tag{2}$$

Then

$$\big(\forall j,\ u(o(x_j)) = v(o(x_j))\big) \quad \Longleftrightarrow \quad \forall x \in A_P,\ u(o(x)) = v(o(x)).$$

A failed equality at any one of the witnesses is a concrete admissible counterexample. Such a spanning family of realized witnesses exists with $m \le k + 1$; this existence statement supplies neither an algorithm to find it nor a proof of coverage for a sampled family.

Proof. Write $u(n) - v(n) = c \cdot (1, n)$ for a coefficient vector $c \in \mathbb{Q}^{k+1}$. If the difference is zero at each witness, the linear functional $z \mapsto c \cdot z$ vanishes on their span and hence on every $\widetilde{o}(x)$ by (2). The reverse implication follows by specializing to each admissible witness. A failed witness equality directly contradicts the universal conclusion.

For the final existence statement, the set of lifted observations spans a subspace of a vector space of dimension $k + 1$. Choose a basis from this spanning set; it has at most $k + 1$ members and each member is a realized observation. If there are no admissible inputs, the span is zero and the empty family suffices. Selecting basis members from an abstract set is a mathematical existence argument; an executable producer needs additional effective information. □

The theorem is elementary linear algebra, not a claim of new linear algebra. Its proposed use is a refinement of the synthesis interface: completeness belongs to the *admissible observation domain*, not merely to the polynomial grammar or the ambient vector space. It extends the affine-length discussion in the previous round without replacing the required source and provider bridges.

9.4 Three concrete observation domains

For unrestricted lists over an inhabited element type with observation $n = \text{length}(xs)$, the empty and singleton lists have lifted observations $(1, 0)$ and $(1, 1)$. Every $(1, n)$ lies in their rational span. Given the grammar's sound affine-length theorem, these two actual witnesses determine an affine scalar length law.

For `List Empty`, the empty list alone spans all realized lifts, since there are no other inputs. The two coefficient vectors for 0 and n differ but induce the same functional on this one-dimensional span. A checker requiring unrestricted coefficient equality is sound when it accepts, but need not accept every true source equality.

For a pair of observations taken from the *same* list, $o(xs) = (n, n)$, the lifts are $(1, n, n)$. The two witnesses with $n = 0, 1$ span the image's affine lift, and the forms n_1 and n_2 agree there. A hypothetical model witness $(n_1, n_2) = (0, 1)$ is not realizable and cannot refute the source equality. The remedy is a joint observation contract, not two separate scalar realizability assertions.

An admissibility guard can change the useful witnesses without changing the grammar. For nonempty lists, lengths 1 and 2 provide a basis for affine laws on the admissible domain; length 0 is not an admissible witness. The coverage theorem is the reusable mathematical package that makes these finite checks universal.

9.5 What this does not authorize

Sampling until the observed rank stops increasing does not prove that a later input will not enlarge the span. The coverage premise is a theorem, not an empirical stopping criterion. Affine equalities also do not characterize sorting: length preservation permits identity and reversal, and sortedness alone permits discarding the input. A sorting specification needs an order property and a permutation or multiset-preservation property, with stability a further requirement when requested.

The same limitation applies to nonlinear or piecewise-affine behavior. A finite spanning set for the inputs determines an affine functional, not an arbitrary function agreeing with it at those points. A larger grammar requires a new soundness or completeness theorem. Positive certificate soundness, completeness of a decision procedure, and realization of counterexamples remain separately stated guarantees.

9.6 Connecting this to Leant requests

A Laurel synthesis request should carry the exact target type, the source behavioral predicate, the admissible input predicate, permitted providers and their proof-linked contracts, and any model-specific soundness or coverage theorem. Leant can search for a term using its existing typed synthesis machinery

and rank candidates using model information. Laurel's acceptance boundary asks for evidence about the exact selected term.

A model counterexample can guide ranking even when it is not yet a source refutation, provided it is labeled model-relative. It cannot justify deleting a mathematically valid candidate from a purportedly complete source search. Refuting one completed candidate also does not refute a sketch with other completions, much less the existence of every possible function of the requested behavior.

The first useful implementation need not solve arbitrary quantified behavioral predicates. It should close one grammar-specific loop: exact syntax to denotation, denotation to observation, abstract certificate to source theorem, and a realized negative witness when rejection is claimed. The fixed affine-length grammar remains a sensible development case, while a fresh nonalgebraic contract should be reserved for transfer evaluation.

10 A verified-CAS interface, not algebra in conversion

10.1 Three computation routes

Laurel should support a proved algorithm, a proved checker of an untrusted certificate, and ordinary proof-producing tactics. Their common output is a result together with a proof of the fixed requested relation to the input. A simplifier may return an equality; a root isolation service may return an enclosure and coverage facts; a solver may return one witness or a complete solution predicate. These are different result types, not levels on one implicit ladder.

The execution route matters independently. A theorem about an executable checker does not by itself certify arbitrary bytes returned by a native process. The strict route reconstructs the proof or reduces the appropriate checker application within the accepted logical boundary. Any accelerated route with additional axioms or trusted code requires an explicit, version-specific policy. The previous round emphasizes this execution distinction; Laurel retains it rather than proposing CAS calls inside definitional equality [1, 2].

10.2 A small affine nonnegativity contract

For real variables x, suppose $g_1(x), \ldots, g_m(x)$ are known nonnegative affine expressions. A certificate for $p(x) \geq 0$ can be rational coefficients $\lambda_i \geq 0$ and an exact polynomial identity

$$p = \sum_{i=1}^{m} \lambda_i g_i.$$

The checker verifies rational coefficient equality and coefficient signs. The soundness theorem is immediate from the identity and closure of nonnegative reals under multiplication and addition. What is not immediate in an implementation is that the checked coefficient expressions denote the author's original terms with the same scalar interpretation and opaque atoms.

For the Fabius derivative bound with $0 \leq u \leq 1$, the obligations $2u \geq 0$ and $2 - 2u \geq 0$ have certificates $2u = 2 \cdot u$ and $2 - 2u = 2(1 - u)$. This is useful as an interface example, but it is not a reason to outperform or replace existing arithmetic tactics without measurement. The accompanying prototype implements residual logic, not this real-expression reifier or its Lean soundness theorem.

10.3 One denotation interface, several reifiers

The round-three question about reification should not be answered by a single untyped universal reifier. An arithmetic AST over a ring, a list-expression AST with an affine-length observation, and a partial operation with a domain proof have different semantics. What can be shared is a typed interface:

$$\mathrm{reify}(t) = (e, \eta, h_t), \qquad h_t : \mathrm{denote}(e, \eta) = t,$$

with a suitable relation in place of equality when the output is an observation rather than an exact representation.

Arithmetic atoms must refer to exact elaborated terms in a fixed environment η. The certificate checker theorem quantifies over every such environment and the needed algebraic structure. The original-target bridge instantiates it with this environment and composes with the reification proofs. Arity, coefficient domain, coercions, and implicit structures are checked before a certificate is accepted. A certificate for a nearby expression is irrelevant even if both expressions look similar when printed.

10.4 Precision and domain are type indices

For formal series, agreement modulo X^N is not equality of the whole series. Differentiation transports agreement modulo X^{N+1} to agreement modulo X^N; it does not preserve the same truncation index by default. A residual-to-root argument needs its own branch or constant-term condition and invertibility hypotheses. For real roots, a square identity alone does not select the nonnegative branch.

These distinctions fit ordinary indexed propositions. A Laurel operation can expose a requirement transformer that asks for enough input precision to achieve a requested output precision. This is a theorem-backed contract, not a new notion of computational equality. Backward demands and forward evidence then meet at a concrete statement such as the needed truncation index or nonvanishing condition.

11 Worked case I: exact Frey quotients

11.1 A satisfiable mathematical package

Use the following assumptions on integers a, b and a natural number p:

$$a \equiv 3 \pmod 4, \qquad 2 \mid b, \qquad p \text{ odd}, \qquad p \geq 4. \tag{3}$$

They are satisfied by $(a, b, p) = (3, 2, 5)$. Unlike a complete contradictory Fermat counterexample package, this helper context is appropriate for testing whether exact-quotient reasoning is really being performed. The distinction is identified in both syntheses; it is not an allegation that the source theorem is incorrect [1, 2].

Define

$$N_2 = b^p - 1 - a^p, \qquad N_4 = -a^p b^p.$$

Proposition 11.1 (Operation-specific exactness)**.** *Under* (3)*,* $4 \mid N_2$ *and* $16 \mid N_4$*. The second conclusion requires only* $2 \mid b$ *and* $p \geq 4$*.*

Proof. Since $b = 2c$ for some integer c and $p \geq 4$, the integer $b^p = 2^p c^p$ is divisible by both 4 and 16. Since $a \equiv -1 \pmod 4$ and p is odd, $a^p \equiv -1 \pmod 4$. Thus $N_2 \equiv 0 - 1 - (-1) = 0 \pmod 4$. Finally, $16 \mid b^p$ implies $16 \mid -a^p b^p$, independently of the parity of p or the residue of a. □

The first exactness conclusion could use a weaker exponent lower bound than $p \geq 4$ if treated on its own. The package uses the common bound for the two coefficients; the registry should still retain the narrower support of each chosen theorem rather than attach every package premise to every operation.

11.2 Transport is a two-stage theorem

Let $n, d, q \in \mathbb{Z}$ satisfy $dq = n$, and let $\phi : \mathbb{Z} \to K$ be a ring homomorphism. Then

$$\phi(d)\phi(q) = \phi(n).$$

This balance transport holds without invertibility. If K is a field and $\phi(d) \neq 0$, it further gives

$$\phi(q) = \frac{\phi(n)}{\phi(d)}.$$

For $K = \mathbb{Q}$, divisibility and $d \neq 0$ identify the integer division result with the exact quotient, and the rational denominator is nonzero. A homomorphism into a ring of positive characteristic may send a nonzero integer denominator to zero, so source nonzeroness cannot replace the destination premise.

Applied to $N_2/4$ and $N_4/16$, these contracts supply the two nontrivial coefficient equalities needed by extensionality of the Weierstrass models. The source uses a rational cast-of-division interface; the abstraction here factors its meaning into exactness and destination interpretation [4].

11.3 Proposed Laurel presentation

```
context FreyArithmetic(a b : Int, p : Nat)
  assume a == 3 (mod 4), even b, odd p, 4 <= p
  let n2 := b^p - 1 - a^p
  let n4 := -(a^p) * b^p

  establish 4 divides n2 by modular_arithmetic
  establish 16 divides n4 by power_divisibility

  let z2 := exact quotient n2 by 4
  let z4 := exact quotient n4 by 16
  establish cast(z2, Rat) = cast(n2, Rat) / 4
    by exact_quotient_transport
  establish cast(z4, Rat) = cast(n4, Rat) / 16
    by exact_quotient_transport
end
```

This is explanatory proposed notation. The actual executable fixture represents the a_4 use site by ground atoms for evenness, the exponent bound, divisibility by 16, rational denominator nonzeroness, and the coefficient identity. With the exponent and rational denominator facts known, and either evenness or divisibility offered, it computes two singleton residual supports. Both have conditional derivations; neither is accepted as an unconditional coefficient equality.

The direct divisibility support and the evenness support serve different authoring purposes. The first names exactly what the consumer needs. The second explains a structural route available in this arithmetic package. Their coexistence is a feature of residualization, not a claim that they are independent mathematical necessities.

11.4 Negative neighbors

At $(3, 2, 5)$ the coefficients are -53 and -486. Changing one package premise gives informative failures:

Triple	Changed premise	Exposed failure
$(3, 2, 4)$	Oddness of p	$N_2 = -66$ is not divisible by 4.
$(3, 3, 5)$	Evenness of b	$N_2 = -1$ is not divisible by 4.
$(3, 2, 3)$	Bound $p \geq 4$	$N_4 = -216$ is not divisible by 16.
$(1, 2, 5)$	Congruence of a	$N_2 = 30$ is not divisible by 4.

These are counterexamples to the specified neighboring arithmetic claims, not failures of Lean casts in general. A missing premise can sometimes be replaced by a different sufficient route. The article's finite arithmetic tests check these values and a small family of admissible triples; Proposition 11.1, not the enumeration, supplies the general argument.

12 Worked case II: sharp Fabius regularity

12.1 A reusable mathematical interface

The regularity source supplies, for its fixed bounded Fabius object and defining property, differentiability and

$$F'(x) = 2U(2x - 1), \qquad 0 \leq U(t) \leq 1, \qquad U(0) = 1.$$

Here F denotes the real-valued Fabius realization and U its associated up function. These are mathematical paraphrases of the relevant interfaces, not a new construction of the functions [3].

The first two facts imply $0 \leq F'(x) \leq 2$, hence $|F'(x)| \leq 2$. The mean-value argument gives

$$|F(x) - F(y)| \leq 2|x - y| \quad (x, y \in \mathbb{R}).$$

At $x = 1/2$, the derivative equation and $U(0) = 1$ give $F'(1/2) = 2$. Any global Lipschitz constant L bounds the absolute derivative wherever the derivative exists: divide the Lipschitz inequality by $|h|$ and pass to the derivative limit. Therefore $L \geq 2$. The upper and lower bounds together prove leastness.

12.2 The generic sharpness theorem

Proposition 12.1 (A derivative-attainment route to sharpness)**.** *Let $f : \mathbb{R} \to \mathbb{R}$ be differentiable everywhere, let $L \geq 0$, and suppose $|f'(x)| \leq L$ for all x. If $|f'(x_0)| = L$ at some point x_0, then L is the least nonnegative global Lipschitz constant of f.*

Proof. For $x < y$, the mean-value theorem on $[x, y]$ gives $c \in (x, y)$ with $f(y) - f(x) = f'(c)(y - x)$. The derivative bound yields the Lipschitz inequality; equality and reversed order cover the other pairs. If K is any Lipschitz constant, the difference quotients at x_0 have absolute value at most K for nonzero increments. Their limit is $f'(x_0)$, so $L = |f'(x_0)| \leq K$. □

Attainment is a sufficient sharpness route, not a necessary one. A function may have a least Lipschitz constant approached by derivatives or secants without attaining it. A different registered method can use an approximating sequence and a lower-bound theorem. Laurel's frontier names the requirements of the selected method, not an exhaustive characterization of all possible proofs.

12.3 Proposed proof at the right mathematical scale

```
show 2 is the least Lipschitz constant of F
proof
  establish abs(D(F)(x)) <= 2 for every real x
    from the derivative equation and 0 <= U <= 1
    by affine_bounds
  establish D(F)(1/2) = 2
    from the derivative equation and U(0) = 1
    by exact_evaluation
  conclude by derivative_attainment_sharpness
```

The source's derivative-certificate conversion, norm normalization, and nonnegative-real coercions belong inside checked adapters. The statement still says *least*, so the second step cannot be silently dropped. A service returning only `LipschitzWith 2 F` has not satisfied the target.

For the up function, the source uses a derivative involving the difference of two values in $[0, 1]$. Its absolute value is at most 2; an appropriate midpoint derivative gives the matching lower bound. The same sharpness interface applies while the function-specific derivative and value theorems remain distinct evidence [3].

12.4 What would count as a successful compression

The full comparison should include ordinary Lean supplied with the generic sharpness theorem, the same derivative interfaces, and the same arithmetic adapters. Laurel's incremental benefit would be fewer explicit routine premises at use sites, more informative failure explanations, or easier repairs when a premise changes. A three-line Laurel proof versus the existing source is not a valid isolated measure of the syntax's value, because much of the reduction comes from the shared mathematical theorem.

13 Worked case III: locality, measures, and quantifiers

13.1 Differentiating an identity

For functions $f, g : \mathbb{R} \to \mathbb{R}$, equality on an open set U and a membership proof $x \in U$ imply equality in a neighborhood of x. A local differentiation consumer can use that relation. Equality only at x cannot: $f(t) = t$ and $g(t) = 0$ agree at 0 but have different derivatives there.

The previous synthesis checked the corresponding Lean locality bridge at its toolchain: open-set membership gives a neighborhood, equality on the set gives eventual equality, and that relation gives equality of derivatives [2]. Laurel should register those as a typed route. A request to differentiate a point equality should remain blocked unless another source of local equality is found. It should not automatically strengthen the relation.

13.2 Almost-everywhere equality has different consumers

An almost-everywhere relation carries a measure. Under the appropriate integral interface, $f = g$ almost everywhere with respect to μ permits equality of their integrals with respect to μ. It does not permit equality at an arbitrarily selected point. Nor can an integrability fact for one measure or a continuity fact for one topology be reused after silently selecting another.

For a concrete negative neighbor, let f be zero and let g be the indicator of the singleton $\{0\}$ on $\mathbb{R}$ with Lebesgue measure. They are equal almost everywhere but $f(0) \neq g(0)$. A row such as "equal almost

everywhere" must therefore be indexed by both the functions and the measure, and point evaluation must not be registered as a consumer of that relation without further hypotheses.

There is also a quantifier trap. For each $t \in [0, 1]$, the predicate $x \neq t$ holds for almost every x in the interval. But it is false that almost every x differs from *every* $t \in [0, 1]$. Countable conjunction has a measure-theoretic route through a countable union of null sets; uncountable conjunction does not inherit that route. The property engine must preserve binder structure, not merge a family of rows by treating quantification as a finite conjunction.

13.3 Interchange requires a route, not a slogan

A step exchanging a limit and an integral needs the hypotheses of a suitable theorem. Dominated convergence, convergence in an integral norm, or a summability theorem can provide different sufficient routes. Pointwise convergence alone does not authorize interchange. Conversely, absence of domination is not a proof that interchange is impossible.

Residualization is useful precisely here. The result can retain a conditional proof under one theorem's domination and measurability premises while also exposing an alternative norm-convergence route. The dependent family, region, measure, and quantifier order remain fixed. The symbolic `alternatives.laurel` example only models the alternative-route logic; it does not encode or verify an analytic interchange theorem.

14 How far should Lean's type system change?

14.1 The question is more than syntax

There are at least three meanings of "extend the type system." One can extend the types users *write*, extend the judgments the elaborator *infers*, or change the terms and conversion rules the kernel *accepts*. Laurel recommends the first two initially, but the third deserves a concrete analysis rather than an automatic refusal.

Refinement types with explicit proof objects have been developed in other settings; Ghalayini and Krishnaswami give a theory with proof-containing refinements and an erasure semantics, with formalized metatheory [12]. This supports taking rich specifications seriously without assuming that an SMT solver must decide every refinement. It does not automatically establish conservativity or implementation correctness for a particular extension of Lean.

14.2 Option A: a library and renderer

The least invasive version is a library of properties, contracts, and adapters plus a read-only mathematical renderer. All requirements are explicit proof arguments in ordinary Lean. This version can already expose a compact interface, retain proof explanations, and produce sensible documentation.

It is the necessary baseline. If it accounts for nearly all benefits, shipping it is a successful outcome. A new language should not survive merely because it has acquired a name or a parser.

14.3 Option B: native author-facing requirement binders

The preferred first extension adds requirement-binder metadata and an elaborator service that resolves such parameters, produces residual contracts, and attaches source locations to proof obligations. The

generated declaration still has ordinary dependent function types and proof arguments. No kernel-level semantic subtype relation is added.

This can be a genuine typing improvement from the user's perspective. A term can be checked against a mathematical contract, not just against its carrier, and method application can infer the missing proof requirements. It is more than pretty printing while remaining conservative at the proof-language boundary. It may initially be exposed as a Lean elaborator extension or tactic command rather than a standalone syntax.

14.4 Option C: an explicit-evidence primitive refinement former

A possible kernel experiment introduces a primitive

$$\mathrm{Ref}(A, P), \qquad A : \mathrm{Type}_u, \quad P : A \to \mathrm{Prop},$$

with introduction $\mathrm{refine}(t, p)$, projection val, and evidence projection property. Its principal rules are

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash p : P(t)}{\Gamma \vdash \mathrm{refine}(t, p) : \mathrm{Ref}(A, P)}, \qquad \frac{\Gamma \vdash r : \mathrm{Ref}(A, P)}{\Gamma \vdash \mathrm{property}(r) : P(\mathrm{val}(r))}.$$

The value projection reduces by

$$\mathrm{val}(\mathrm{refine}(t, p)) \longmapsto t.$$

Subsumption is explicit in evidence: given $h : \forall x, P(x) \to Q(x)$, weakening produces a refined value with the same underlying t and proof $h(t)(p)$. It is not a rule that changes a type merely because the proposition looks stronger.

There is an evident proposed translation into ordinary subtypes,

$$\mathrm{Ref}(A, P) \mapsto \{x : A \mid P(x)\}, \qquad \mathrm{refine}(t, p) \mapsto \langle t, p \rangle.$$

For the explicit introduction and projection fragment, typing and the displayed reduction are preserved by this translation. That observation makes the primitive plausible as a conservative implementation experiment. It is not a completed conservativity proof for an extension interacting with every Lean feature.

A serious experiment must specify its universe behavior, eliminators, equality and structure-eta behavior, interaction with proof irrelevance, nested dependent types, quotient operations, and compilation erasure. It must establish that dependent uses of the projected value behave as intended. If it adds conversion principles not simulated by the translation, it needs a new argument, not just the word "refinement."

14.5 What a primitive could and could not buy

A primitive might reduce overhead for proof-erased views, projection-heavy interfaces, or certain dependent coercions. It could provide a stable internal representation for author-facing contracts. These are hypotheses to profile against optimized ordinary subtype and structure encodings, not demonstrated benefits.

It cannot infer missing mathematical evidence by itself. The need to prove a denominator invertible, a function measurable, a representation natural, or a witness admissible remains. If most author effort is theorem search and obligation presentation, a primitive will not address the main bottleneck. A kernel experiment should begin only after measurements identify a specific representation or checking cost that the elaborator/library approach cannot remove.

The Lean4Lean work shows why kernel changes need a careful specification and verification story rather than confidence based on a small rule diagram. It develops an external checker and reports formalization and verification of parts of the kernel, not a blanket theorem making arbitrary extensions safe [15].

14.6 Option D: semantic subtyping or equality reflection

A substantially different proposal would allow conversion to ask whether arbitrary $P \Rightarrow Q$ or $s = t$ is provable. Laurel does not recommend this. It would put theorem search, solver policy, and potentially external computation into an especially sensitive checking boundary. Failure and timeout would need their own semantics, and old elaboration behavior could become dependent on a changing search portfolio.

The objection should not be oversold as preserving a simplistic theorem that all existing Lean checking is a terminating decision procedure. Conversion and its implementations already have subtle metatheory and resource behavior. The narrower, defensible claim is that proof-producing elaboration preserves the existing trusted boundary and makes the additional search explicitly inspectable. It does not require arbitrary mathematical implication to become conversion.

A restricted, proved normalization procedure can still be used. It should return evidence through a checker interface or be proposed as a separately justified conversion extension with its own metatheory. The former is the initial Laurel route.

14.7 A concrete stopping rule for the kernel experiment

Before trying a primitive, run the same proof corpus with explicit arguments, ordinary subtype packages, anchored evidence plus generated arguments, and an optimized renderer. Measure elaboration time, kernel-checking time, term size, projection/transport frequency, and author interventions. Only a persistent, attributable cost in the representation or checking layer motivates a new kernel primitive. Even then, accept it only with a translation or metatheory, negative tests, and independent checking support appropriate to the new rules.

15 Trust, methods, and faithful mathematical explanation

15.1 Logical truth and statement fidelity are separate checks

A kernel can verify a correct proof of the wrong statement. Laurel therefore needs a frozen semantic request and a source-to-evidence map. Before accepting a producer's output, check the exact elaborated proposition, its structure arguments, and its permitted metavariable assignments. Before rendering it, check that the displayed claim has the same quantifiers, domain, relation, and strength.

This is especially important for claims such as "complete solution set," "least constant," or "everywhere differentiable." Returning a witness, an upper bound, or an almost-everywhere result may be mathematically valuable but does not complete those targets. A suggestion to weaken the claim must be an explicit edit, not a successful elaboration hidden behind concise prose.

15.2 Required methods are not arbitrary hints

If an author explicitly says `by exact_quotient_transport`, the accepted derivation should instantiate that method's interface and discharge its premises. A proof by an unrelated global contradiction may prove the same proposition, but it does not explain the advertised arithmetic route. Conversely, `by?` can deliberately delegate method selection.

Checking that a theorem's name occurs somewhere in a proof is not sufficient method fidelity. A useful implementation retains a small method derivation tree with admissible subproducers and compares it to the selected interface. It also restricts which contextual facts are available to the route when the author

or benchmark requests that restriction. The semantic proof still goes through the ordinary kernel; the method record checks an additional explanatory contract.

For this reason, the finite algorithm's supports should be computed separately for distinct required-method profiles, or be indexed by method identity as well as proposition. Merging two proofs solely because their residual sets agree could erase the method evidence the author asked to preserve. The executed core has one fixed profile at a time and no sophisticated method-language interpreter.

15.3 No hidden publication debt

A document can contain an open research sketch with explicitly pending statements. It must not be presented as a completely checked theorem document until every displayed assertion in that claimed scope has evidence. In particular, unused false assertions cannot be excused because the final theorem's proof term ignores them.

A published theorem should carry its elaborated statement, retained proof or reconstructible certificate, imported revision information, and an axiom-policy result appropriate to its actual environment. The receipt says which checks ran; the proof supplies the logical evidence. Migration to another toolchain produces a new check and a new receipt, not a claim that a previous hash proves compatibility.

15.4 Reproducibility without freezing every search decision

Search order and ranking may change. Reproducible verification should primarily retain accepted proof terms or certificates and their exact statements, rather than requiring the same heuristic search to rediscover the same path. A source suggestion has an additional requirement: replay the exact displayed edit in the recorded context before calling that suggestion verified.

The evidence index is an optimization over the context, not a second independent logic. The prototype's fingerprints ensure exact profile comparison inside the model; they do not prove theorem truth, context equivalence, or authenticity of an external service. Real integration must reconstruct and check the actual expressions.

16 What the executable companion establishes

16.1 Scope and architecture

The archive contains `prototype/laurel_core.py`, a standard-library Python implementation of the finite profile language. It parses named atoms, known facts, offered residuals, Horn rules, and one target; computes antichain frontiers; retains immutable derivation nodes; checks each result with a separate derivation checker; and emits a JSON summary and optional ordinary-Lean proof skeletons. The narrow lost-premise transformation is also included.

The checker does not trust the frontier algorithm. It verifies leaf availability, offered-premise authorization, exact rule membership, premise order, conclusion, target identity, profile identity, and recomputed support. It rejects cyclic proof objects. This is independent checking of a *symbolic Horn derivation*, not Lean-kernel checking or verification of a theorem registry.

The parser accepts a small symbolic DSL. It does not parse mathematical expressions, infer carrier types, select structures, construct witnesses, or inspect Lean declarations. Its rule names are explicit assumptions of the symbolic theory. The generated Lean declarations therefore quantify over their atom propositions and rule implications. No generated declaration asserts that the named Frey or analysis rules hold merely because those names appeared in an input file.

16.2 The implemented use-site example

This is actual executable Laurel-Core syntax:

```
context frey_a4
atoms even_b large_p div16 cast16_nonzero coefficient_identity
known large_p cast16_nonzero
offer even_b div16
rule power_divisibility : even_b large_p -> div16
rule quotient_transport : div16 cast16_nonzero -> coefficient_identity
show coefficient_identity
```

Its complete output contains the two supports {`div16`} and {`even_b`}. The exporter creates one conditional declaration for each. For the evenness route, the logical shape is

$$(E \to B \to D) \to (D \to Z \to C) \to Z \to B \to E \to C.$$

Here E, B, D, Z, C are arbitrary propositions standing for the fixture's atoms. The proof applies the first rule to E, B and the second to the resulting D, Z. This is a real generated proof term shape, but not a Lean verification of the mathematical interpretation of those letters.

16.3 Exhaustive and adversarial checks

The exhaustive oracle does not use antichains to derive its answer. It enumerates every subset of offered assumptions, computes ordinary Boolean Horn closure, and then selects inclusion-minimal successful subsets. With three atoms, there are nine possible nonempty-premise rules whose conclusion is not among their one or two premises. The test enumerates all 512 subsets of these rules, all 8 known-fact sets, and all 8 offered sets.

Executed check	Count	What was compared or checked
Exhaustive profiles	32,768	Every rule subset and every known/offered subset in the three-atom family.
Exhaustive target frontiers	98,304	Three targets per profile against the independent closure oracle.
Exhaustive root derivations	98,400	Every returned support's derivation checked independently.
Randomized profiles	500	Six atoms, variable arity, self-premises and empty-premise rules possible.
Randomized target frontiers	3,000	Six targets per profile against the oracle.
Randomized root derivations	2,527	Returned derivations checked; budgeted versions of all 500 profiles also checked.
Boundary test methods	19	Scope, corruption, target identity, conditional status, syntax, budgets, and repair.
Admissible Frey triples	308	Finite arithmetic checks in explicitly bounded ranges.
Single-premise Frey mutations	4	The four values in Section 11.

Table 2: Executed evidence. None of these counts denotes Lean compilations or human proof-authoring tasks.

All listed tests passed in the retained run. The JSON records Python 3.13.5 and the platform; the combined test script took about 2.4 seconds in that one local run. This is a reproducibility detail, not a claim of advantage over Lean. The exhaustive family is tiny and excludes some rule forms that the targeted and randomized tests cover; it is exhaustive only in the precisely stated family.

The boundary tests include an unseeded cycle, a nonempty support incorrectly advertised as empty, a changed target, a changed rule profile, a sibling context, a forged known leaf, a missing rule, reversed premise order, a zero work budget, and repair after removal of used versus unused facts. The counterexample model checks include the exact image $\{0\}$ for an empty element type and the incompatibility of two observations purporting to come from one input. The latter are small model fixtures, not compiled source-level realization theorems.

16.4 An experiment that exhibits the bottleneck

The width family from Section 6 was run separately. At $k = 2, 4, 6, 8, 10$, the final widths were respectively $4, 16, 64, 256, 1024$. The log records one local time per case and the number of premise combinations attempted. This confirms the expected growth for that family and demonstrates why a complete alternative frontier cannot have a uniformly small representation as an explicit list.

This result is more relevant to design than the tiny test-suite runtime. It supports making complete and partial modes explicit and avoiding a promise that Laurel always returns every useful explanation interactively. The proof validity of a returned route need not depend on exhaustiveness.

16.5 What remains unverified

The tests are not a proof of the Python implementation's correctness. The oracle and engine were written for this work and share the same mathematical specification; independent implementation review would be stronger evidence. The written soundness and completeness theorems have not been mechanized here.

The Lean exporter was exercised, and its generated files contain explicit hypotheses and proof applications without `sorry` or introduced axiom declarations. They were not compiled. The full frontend's meaning-hole discipline, dependent context transport, theorem-registration checks, certificate reification, interaction with Lean's elaboration state, and reader-facing renderer are not implemented. No compression, repair-time, or productivity number is claimed for real mathematical authoring.

17 An implementation plan with exit conditions

17.1 First deliverable: one property-aware Lean use site

The first Lean artifact should be a small package extending the already checked helper work reported by the syntheses, not a broad standalone grammar. Expose a proof-implicit exact-quotient operation, a validated registry for the necessary divisibility and cast interfaces, and a residual result type carrying a proof continuation. Run it on a satisfiable Frey arithmetic context with the four negative neighbors.

The acceptance gate is not that it can print a short proof. It must produce a proof of the exact original statement when complete; retain conditional evidence when a premise is missing; reject unauthorized statement changes; and check the resulting declarations on one pinned toolchain. The symbolic implementation here supplies the algorithmic specification for the finite support part, not a substitute for that gate.

17.2 Second deliverable: locality and context rebuilding

Add the open-set-to-neighborhood differentiation interface. Test both a valid local equality and a point-only equality. Then rebuild the surrounding Lean declaration after edits, change local identifiers,

and exercise `simp` on the anchored expression. This stage is where the model's string identities must be replaced by actual expressions and typed context information.

The initial lifetime policy is conservative: rebuild the index per declaration; retain proof objects only through checked re-elaboration or explicit context embeddings. Optimize persistence after measuring that rebuilding matters. A speculative cross-session proof cache is not the smallest next step.

17.3 Third deliverable: a proof-linked Leant contract

Select a fixed list-expression grammar and formalize its observation theorem. Register exact provider laws as Lean theorems. Implement either the unrestricted inhabited-domain affine criterion or the realized-span variant for a guarded/correlated observation domain. For acceptance, export a proof about the exact candidate; for refutation, export an actual admissible witness and a proof of failure.

The test pair `List Nat`/`List Empty` should have different completeness behavior without any heuristic exception keyed to those printed type names. The distinction must arise from the registered realization or coverage theorem. A changed candidate term must invalidate a behavioral receipt for the old term.

17.4 Fourth deliverable: publication and a minimal surface

Only after the use-site interfaces work should the narrative syntax be added. The renderer can be developed earlier, but it must consume the same typed evidence model rather than infer mathematics independently from a prose string. A compact assertion and its expanded version must share a statement identity and evidence record.

At this point, run a reading test in which users recover the assumptions, domain, quantifiers, and guarantee from the compact view before opening the expanded proof. The purpose is to discover misleading omissions, not merely to ask whether readers like the syntax. A library-only or renderer-only product remains a legitimate outcome.

18 Evaluation that can falsify the proposal

18.1 Equalize libraries and services

Use at least the following comparison arms: ordinary Lean; Lean with the new mathematical packages and adapters; Lean with those packages plus the same added search and certificate services; property-aware elaboration over the same resources; and the narrative Laurel surface over that same elaborator. A renderer-only control can isolate the value of presentation.

Existing automation belongs in the ordinary-Lean baseline. New reusable theorems and wrappers belong in every arm that is supposed to share the library. Leant or a CAS must not be available only to the Laurel arm and then have its contribution attributed to syntax. The relevant incremental comparison is between equally equipped authoring interfaces.

18.2 Measure author work and semantic recovery

Primary outcomes should include successful completion of the exact statement, author interventions, time spent locating and supplying side conditions, recovery after controlled edits, and correct reading of compact claims. Record failures separately: missing mathematical facts, unsupported grammar, exhausted budget, ambiguous structure selection, and actual counterexamples are not one category.

System measurements should include elaboration and kernel times, generated proof size, index size, maximum frontier width, rule-instance generation, and the proportion of solved obligations that were single-lemma lookups. These measurements answer whether residualization adds value or merely repeats an existing tactic in a new subsystem.

The present examples are development cases because they were inspected while designing Laurel. Held-out tasks must come from declarations not used to construct the wrappers, rules, or tests. A transfer set should contain at least one domain where the salient property is not affine arithmetic, such as a topological restriction or a measure-theoretic consumer.

18.3 Predeclared interpretations of outcomes

If the shared-library arm explains the gains, keep the library. If a renderer explains the comprehension benefit, keep the renderer. If property-aware elaboration reduces repair or side-condition work beyond equally equipped Lean, retain that layer even if users prefer ordinary Lean syntax. If the full surface helps experts but systematically misleads occasional readers about hypotheses, revise its defaults rather than presenting compression alone as success.

No numerical success threshold is asserted here without pilot data. A study should preregister its primary outcome, sampling plan, and interpretation before collecting confirmatory results. Line counts can be descriptive, but they are not a substitute for successful authoring or faithful reading.

19 Answers to the round-three implementation questions

The syntheses ask overlapping questions at different granularities. Table 3 gives a concrete response to the second report's ten fourth-round questions; the following paragraphs connect the first report's principal challenges to the same decisions. This is an implementation crosswalk, not another feature vote [1, 2].

Table 3: Fourth-round question crosswalk. "Measured" is reserved for the symbolic experiments actually run here.

Id	Question	Laurel's answer and status
T1	Cost of the index on a real file	Not measured. Instrument one Lean exact-quotient use site before generalizing. Symbolic width growth is measured, but is not a real-file index measurement.
T2	Which rules actually fire?	Retain rule IDs in proof DAGs; distinguish lookup from composition. The fixture uses power-divisibility and quotient-transport routes. Real corpus frequencies remain to be collected.
T3	Does simplification break anchors?	Use conversion or a checked equality/property transport; retain exact structures. Rebuild per declaration initially. No string-based migration.
T4	One reifier for several checkers?	Share a typed denotation/bridge interface. Arithmetic and observation grammars need separate semantics; a universal untyped reifier is not proposed.
T5	Completeness of behavioral abstractions?	State it relative to the admissible observation image. Theorem 9.1 gives an affine realized-span criterion with explicit coverage; general Leant contracts remain separate cases.
T6	Almost-everywhere rows and consumers?	Index the relation by its measure and use a registered integral consumer. Do not provide point evaluation or uncountable conjunction as default consumers.

Id	Question	Laurel's answer and status
T7	Shorter FLT structure preambles?	A lexical structure selection policy is proposed, not measured. Do not infer a benefit from the exact-quotient example, which is a different cost.
T8	Bounded rule generation?	Finite typed term pool, registered patterns, bounded fresh-term depth and instance counts; completeness is relative to the resulting finite profile.
T9	What does a reader recover?	Not measured. Run a blind semantic-recovery test on fixed claims before allowing the expanded view. No readability claim follows from the prototype.
T10	Independent implementations?	The archive gives a reference engine and a different-algorithm oracle, not independent teams. A separate Lean implementation and independent adversarial review are the next stronger checks.

The first synthesis's insistence on a *first real use site* is answered by the exact-quotient binder, not by the full prose grammar. Its concern about finite grounding is addressed in Section 7. Authorized data choice is isolated from proof search in Section 5; a required method is represented as an explanatory contract in Section 15. The source-behavior bridge and sound negative outcomes are specified in Section 9, while Section 13 supplies a nonalgebraic development transfer.

Replay is based on accepted proof artifacts and destination checks rather than a promise of identical search. Native execution remains a separately audited policy, with no unmeasured crossover threshold. Finally, the stopping rule is explicit: retain only the layers that add value beyond equally equipped Lean. These answers are concrete decisions where possible and named experiments where evidence is genuinely absent.

20 Risks and limitations of the design

20.1 The second-elaborator risk

An evidence index can become an expensive duplicate of Lean's elaborator. Laurel avoids claiming a new universal inference engine: it has a small finite profile, typed interfaces to existing producers, and explicit limits. Even so, integration can duplicate instance resolution, equality normalization, and context maintenance. Instrumentation and the library-only baseline are essential safeguards against solving the same problem twice.

20.2 Explanations can be correct but unhelpful

A frontier may offer too many syntactically incomparable sufficient conditions. Minimality by subset inclusion does not ensure mathematical relevance. It may also return a condition essentially equivalent to the original goal. Ranking by author effort, available evidence, or familiar method could help, but those rankings are heuristics. They may reorder or hide routes without changing their validity; they cannot manufacture necessity or completeness.

A concise explanation can also be technically accurate yet misleading. Saying "exact quotient" without revealing the coefficient ring or saying "equal" when the relation is almost-everywhere equality loses essential information. The renderer must privilege guarantee fidelity over a fixed line budget.

20.3 Dependent data remain the hard boundary

The finite model deliberately avoids higher-order unification, open witness choice, universe inference, and dependent context transport. Those are major sources of practical elaboration difficulty. The existence of a finite symbolic algorithm is not evidence that the complete frontend will be predictable. Its contribution is a precise component contract: after grounding and freezing meaning, here is what residual inference computes and what a partial result means.

For the realized-span proposal, proving coverage can be harder than proving the original behavioral law. The approach is worthwhile where many candidates share the same grammar and admissible observation domain, allowing the coverage proof to be reused. It is not a general replacement for direct proof.

20.4 The possibility that Laurel should remain a library

The design is compatible with an outcome in which no separate language is warranted. Existing theorem APIs, automation, and an evidence-aware editor might provide nearly all benefits. A named proposal should remain a hypothesis about authoring, not a commitment to replace a successful host language. Its strongest deliverables should be reusable even if the surface is abandoned.

21 Conclusion

Laurel proposes that the unit of mathematical proof authoring be an *operation on fixed objects under a composable contract*. Properties enrich an object's usable interface without silently replacing its value or structures. Behavioral laws constrain the exact synthesized function. A step that cannot close returns conditional evidence with explicit residual requirements, rather than either guessing or discarding all progress.

The fourth-round increment is concrete. The finite residual frontier has a precise semantics, a terminating proof-producing algorithm, a relative completeness theorem, a separately checked implementation, and tests that exhibit both correct behavior and an exponential output family. Lost hypotheses can be reopened in a retained derivation. Affine behavioral reasoning can be made sensitive to the span of realizable observations, with coverage and refutation obligations stated separately.

These results do not establish a finished Lean frontend or a productivity advantage. They do provide a sharply defined next experiment: implement one proof-implicit exact-quotient use site in Lean, retain its conditional proofs, test its negative neighbors and repairs, and compare it with equally equipped ordinary Lean. The recommended first extension changes the author-facing typing discipline; a native refinement primitive remains a concrete, gated research option. The ultimate criterion is not how little text Laurel can display, but how much mathematical intent it can preserve while reliably removing work the author did not need to do by hand.

A Source register and evidence boundaries

The bibliography links immutable repository files where possible. The archive's `sources.json` records full revisions. Repository source observations, historical reports of compilation, and experiments executed for this article are deliberately separated.

Source	Inspected material	Boundary of the observation
Round-three synthesis A	`docs/round-3/synthesis/unified-report.tex`	Main design, review, evaluation, questions, and reconciliation read. Its experiments are reported evidence, not rerun here.
Round-three synthesis B	`docs/round-3/unified_report/unified_report.tex`	Main text, commitment matrix, assessment, corrections, and fourth-round questions read. Compiled-core and supply claims remain attributed to the report.
ProveIt	`Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean`	The regularity module read directly. No local build of ProveIt was performed.
FLT	`P2M/Sol/S_FreyPackage_freyCurveInt_map.lean`	The coefficient transport and supporting arithmetic source read directly through the solution declaration. No FLT build or independent axiom audit was performed.
Leant documentation	README and opening architecture sections of `docs/synth-internals.md`	Synthesis boundaries read; historical acceptance figures in the documentation were not repeated as new tests.
Leant source	`src/Leant/Synth/Length/Contract.hs`	Full passive contract vocabulary inspected, including exact provider laws and paired-output contract data.
Lean and Mathlib documentation	Type-system reference; `mvcgen`; `fun_prop`; Lean4Lean paper	Used to establish host capabilities and motivate trust boundaries, not to claim the companion was compiled against the latest releases.
Laurel companion	`prototype/` and `evidence/`	Executed here. Scope is the finite symbolic model and listed arithmetic fixtures.

The reconciled sections of the syntheses are important. In particular, positive soundness does not require decision completeness; differing unrestricted length coefficients do not refute a law on `List Empty`; and shared semantic patterns do not make all earlier Lean encodings syntactically identical. Laurel follows those corrected distinctions rather than earlier shorthand in the review narrative [1, 2].

B Reproducing the companion

From the extracted archive directory:

```
cd prototype
python test_core.py
python bench_frontier.py
```

```
python laurel_core.py examples/frey_a4.laurel \
  --json generated/frey_a4.json \
  --lean generated/frey_a4.lean
python laurel_core.py examples/quotient.laurel --budget 0
```

The scripts need Python 3.10 or later and only its standard library. Tests were run with Python 3.13.5. A work cap produces a partial result rather than a mathematical negation. The generated JSON distinguishes a conditional route from a result established relative to the symbolic input assumptions.

To rebuild this article, run `latexmk -pdf Laurel.tex` in the archive root, or run `pdflatex Laurel.tex` sufficiently many times to settle references and the contents. The LaTeX source is self-contained; it does not require downloading repository sources or executing the prototype during typesetting. A standard TeX installation with the packages named in the preamble is required.

A future Lean check can compile each generated `.lean` file in a chosen Lean environment. These declarations use only propositions, implications, hypotheses, and local theorem applications, but that inspection is not a replacement for actually running the checker. The archive labels them uncompiled. No external repository or user account is modified by the companion.

C A compact rejection suite for the intended Lean layer

The following cases extend the symbolic tests into acceptance requirements for an actual implementation. They are specifications for future tests, not a claim that the Lean layer has already rejected them.

Positive case	Neighboring mutation	Required outcome
Exact quotient with divisibility	Cast integer $1/4$ as rational $1/4$	Reject the equality route; preserve the actual integer division meaning.
Denominator a unit in the target	Denominator merely nonzero in the source ring	Expose the missing target condition, not an invented inverse.
Neighborhood equality	Equality only at the evaluation point	Reject differentiation through that relation.
Almost-everywhere equality for μ	Point evaluation or a different measure	Require an appropriate new theorem or reject the consumer.
Countable a.e. family	Uncountable conjunction	Do not merge the family by the countable rule.
Lipschitz bound plus lower bound	Upper bound alone requested as least	Keep leastness unfinished.
Jet agreement modulo X^{N+1}	Differentiate and retain the same order	Require the correct precision transformation.
A witnessed complete solution set	Only coverage, admitting spurious solutions	Reject completeness without soundness.
List length law on inhabited type	Unrealizable positive length for `List Empty`	Do not issue a source refutation.
Joint observation from one input	Independently realizable but incompatible marginals	Require one simultaneous realizing input.
Exact provider theorem attached	Passive law text only	Treat the law as unproved, not as evidence.
Fixed function under a contract	Different well-typed candidate substituted afterward	Invalidate the old behavioral evidence.
Fact in one branch	Reuse in a sibling branch	Reject or export a correctly scoped implication.
Explicitly related rewritten anchor	Same printed spelling after a structure change	Require typed transport, not string equality.

Positive case	Neighboring mutation	Required outcome
Exact requested theorem proved	Nearby weaker proposition proved instead	Reject target mismatch; offer an explicit statement edit.
Named arithmetic method	Global contradiction used in its place	Truth may be checked, but the required method contract is unsatisfied.
Every displayed assertion checked	An unused unsupported assertion included	Do not mark the document fully checked.
No route in the selected fragment	Announce the mathematical claim false	Reject the status promotion.

References

[1] Codex. *From Properties to Proof Obligations: A critical synthesis of the nine round-3 proposals*. Brainstorm, 6 September 2026, revision `bf3333905995060870f8585e297ffc16f3fe7e86`. Pinned LaTeX source: synthesis/unified-report.tex.

[2] Claude (Anthropic). *Nine Refinements: Property-Aware Typing in a Mathematician's Proof Language over Lean*. Brainstorm, 6 September 2026, same revision as above. Pinned LaTeX source: unified_report/unified_report.tex.

[3] Vladimir Reshetnikov and contributors. *ProveIt: Regularity.lean*, Fabius function development. Revision `24ce8bd743eaab64a91ce90725ea00f498d319d2`. Pinned regularity source.

[4] Anthropic and repository contributors. *Fermat's Last Theorem: S_FreyPackage_freyCurveInt_map.lean*. Revision `aa2d8b34692b16c70f699536de0d8e75b9a3e9ef`. Pinned Frey arithmetic source.

[5] Anthropic and repository contributors. *Fermat's Last Theorem repository README*, same revision. Pinned README. Build and validation statements in this source are the repository's reports, not new checks in this article.

[6] Vladimir Reshetnikov and contributors. *Leant README*. Revision `3a40904be8a410d832d9ab6900b3d3e7b425eccb`. Pinned README.

[7] Vladimir Reshetnikov and contributors. *Leant: :synth internals*, same revision. Pinned synthesis-internals documentation.

[8] Vladimir Reshetnikov and contributors. *Leant.Synth.Length.Contract*, same revision. Pinned Haskell source.

[9] Lean project. *The Lean Language Reference: The Type System*. Documentation consulted 6 September 2026; the latest manual then identified its version as 4.34.0-rc2. `https://lean-lang.org/doc/reference/latest/The-Type-System/`.

[10] Mathlib contributors. *Mathlib.Tactic.FunProp*, generated documentation. Consulted 6 September 2026. `https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/FunProp.html`.

[11] Lean project. *The Lean Language Reference: The mvcgen tactic*. Consulted 6 September 2026. `https://lean-lang.org/doc/reference/latest/The--mvcgen--tactic/`.

[12] Jad Elkhaleq Ghalayini and Neel Krishnaswami. *Explicit Refinement Types*. Proceedings of the ACM on Programming Languages 7 (ICFP), article 195, 2023. DOI: 10.1145/3607837. `https://arxiv.org/abs/2311.13995`.

[13] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. Proceedings of PODS 2007, pp. 31–40. DOI: 10.1145/1265530.1265535.

[14] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*, author presentation at Principles of Provenance, Philadelphia, 26 June 2007. `https://www.cis.upenn.edu/~plclub/propr/greg-slides.pdf`. Used for the distinction between alternative and joint provenance, not as a source of Laurel's specific antichain theorem.

[15] Mario Carneiro. *Lean4Lean: Verifying a Typechecker for Lean, in Lean*. arXiv:2403.14064v3, 14 September 2025. `https://arxiv.org/html/2403.14064v3`.